

操作系统个人知识梳理

1 概述

OS概念

- 一个系统软件，管理控制计算机软硬件资源。合理组织计算机工作流程，提供给用户工作环境。是用户和计算机的接口。
- 向上服务，向下管理

OS特征

- **并发**：同一时间间隔内发生
- **共享**：多个用户使用
- **虚拟**：物理一个实体 -> 逻辑多个
- **异步**：进程各自独立

OS设计目标

- ch1
 - 方便性(方便用户使用)
 - 有效性(使用硬件高效)
- ch3
 - 用户
 - OS应该便于使用(方便性)
 - 易于学习
 - 可靠
 - 安全与快速
 - 系统
 - 易设计、实现和维护
 - 应该灵活、可靠、没有错误且高效

组成部分

- 处理机管理
- 存储器管理
- 设备管理
- 文件管理
- 提供用户接口

内核技术的发展

操作系统的内核



注意:

操作系统**内核**需要运行在**内核态**

操作系统的**非内核**功能运行在**用户态**

微内核

- 只把CPU管理、内存管理等一些基本功能放进内核
- 微内核结构是以微内核为os核心，以客户/服务器为基础，采用面向对象程序设计特征，是当今最有发展前途的OS结构

OS发展阶段

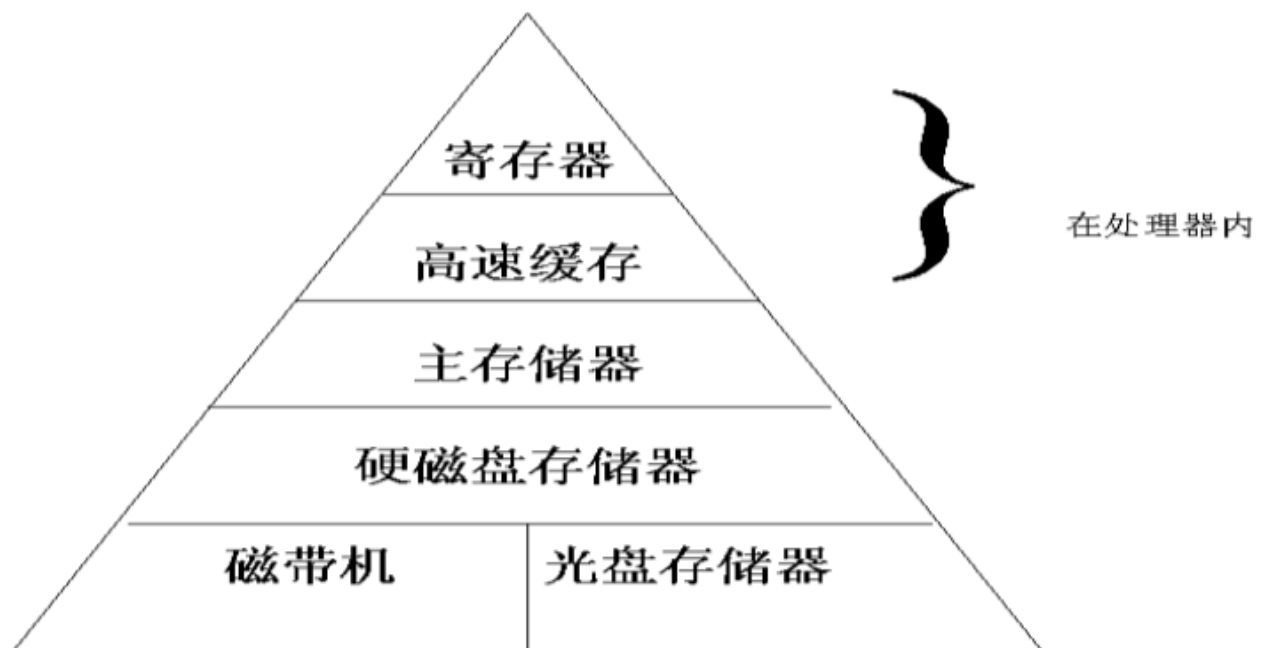
- 批处理（多道程序并发执行：提高资源利用率）
- 分时（人机交互）
- 实时（及时、可靠）

各个操作系统的特点

- 通用操作系统：兼有以下多个功能的OS
- 实时操作系统
 - 3个：航空订票、过程控制、激光照排系统
- 多道批处理操作系统
 - 硬件支持是**通道和中断**
 - 批处理作业**必须有作业控制信息**
 - 不允许用户随时干预自己的程序运行
- 分时操作系统
 - 用户也不可以独占计算机资源
 - 响应可能不太及时
 - 允许一台电脑上连接多个终端，多个用户同时交互使用计算机
 - 时间片一定，用户数越多，响应时间越长
 - **时间片轮转**策略为用户服务
 - 三个特征

- 多路性
- 交互性
- 独占性
- 响应时间
 - 定义：终端用户发出一条命令开始，到系统处理完这条命令并做出回答为止所需的最大时间间隔

2 计算机体系结构



- 性能比：速度、容量、价格

缓存

- 利用高速存储器保存最近访问过的数据
- 解决部件之间速度不匹配

4种 I/O方式

- 可编程I/O **PIO**
 - CPU定期轮询
 - 占用CPU时间：查询 + 传输
- 中断机制
 - 中断：操作系统内核夺回CPU使用权
 - CPU不主动查询外设状态了，让外设通知CPU(中断请求)
 - **中断控制器**处理中断请求
 - 实现CPU与设备、设备间的并行，CPU利用率高
- 直接内存存取 **DMA**
 - DMA控制器
 - 虽然有中断但中断过程变小了，减少时间
 - 提高处理I/O的性能
- 通道技术

- 多出来一个CPU专门用来处理I/O
- 提高CPU与设备、设备间的并行度
- 贵

3 进程管理

- 进程：程序及其数据在计算机的一次执行活动

进程与程序区别

- 进程 = 程序 + 数据
- 进程有生命周期；而程序是相对长久的；
- 进程动态，程序静态；
- 进程可创建其他进程，程序没有；
- 进程描述并发，程序不能。

进程和线程的区别

- 调度性：进程是拥有资源的基本单位。线程是调度的基本单位，线程切换代价更小
- 并发性：进程可以并发执行,一个进程的多个线程也可并发执行。线程更好并发性，**提高系统资源利用率和吞吐率**
- 拥有资源：进程是拥有资源的基本单位,线程本身基本不拥有系统资源，但可以访问隶属进程的资源。
- 系统开销：操作系统在创建、撤消和切换进程时付出的开销显著大于线程。

进程组成

PCB + 程序段 + 数据段

PCB的含义

描述进程的基本情况、运行状态

进程特征

- 并发性
- 动态性
- 独立性
- 交互性
- 异步性
- 结构性

进程状态迁移图 引起迁移的原因和事件

- 创建、就绪、运行、阻塞、终止
- 就绪 -> 运行：进程被调度
- 运行 -> 就绪：时间片到、CPU被更高优先级的进程抢占
- 运行 -> 阻塞：等待资源分配、等待时间发生（主动）
- 阻塞 -> 运行：资源分配到位、时间发生（被动）

进程的创建

- 原语：由操作系统提供的一种不可分割、中断的系统操作
- 创建方式：
 1. 系统模块统一创建
 2. 父进程创建子进程 父进程：创建新进程的进程；子进程：由父进程创建的新进程。子进程可以继承父进程部分资源，有自己独立地址
- 创建事件：用户登录、提供服务、应用请求、作业调度
- 如何创建： 1.申请并创建PCB 2.分配PID（进程标识符） 3.为进程映象分配空间 4.初始化PCB：程序入口地址... 5.设置相应的链接：把新进程加到就绪队列的链表中

进程之间的关系

- 直接（相交进程）：
 - 通信
 - 低级：状态和整数值
 - 高级：传送任意数量的数据。
 - 共享存储区
 - 消息缓冲（消息头、消息体）（发送、接受原语）
 - 管道
 - 同步：进程协同完成任务，**等待**
- 间接（相交、无关进程）：
 - 互斥：竞争**临界资源**（只允许一个进程使用） **临界区**：进程访问临界资源的代码段

高级通信方式 send() receive()的工作过程

- send()：将发送进程的数据copy然后**封装规范化**为数据结构'消息'，存储在内存**系统区**
- receive()：将系统区的消息中的数据copy给接收进程

三个层面的调度

- 高级调度（作业）
- 中级调度（内存）：挂起态（暂时不用的进程放入外存）
- 低级调度（进程）

调度模式

- 非剥夺方式 简单；开销小；实时性差
- 剥夺方式 优先级原则 时间片原则

调度准则

- $$\text{CPU利用率} = \frac{\text{CPU运行时间}}{\text{总时间}} \times 100$$
- $$\text{吞吐量} = \frac{\text{完成的进程数量}}{\text{时间}}$$
- $$\text{等待时间} = \text{开始执行时间} - \text{提交时间}$$
- $$\text{周转时间} = \text{完成时间} - \text{提交时间}$$

$$\text{带权周转时间} = \frac{\text{周转时间}}{\text{运行时间}}$$

- 响应时间 = 第一次获得 CPU 的时间 - 提交时间

常用的调度算法

- 先来先服务FCFS
- 短作业优先SJF (平均等待时间最短)
- 优先级调度 (实时)
- 多级队列调度 (分时)
- 多级反馈队列调度

多级反馈队列调度算法

- 队列间优先级, 队列内FCFS
- 时间片到, 进程进入下一级队列队尾
- 被抢占, 进程进入当前队列队尾

临界资源的使用原则

- 有空让进
- 无空等待
- 多中择一
- 有限等待
- 让权等待 最少资源数: 进程 * (资源 - 1) + 1

互斥的软硬件方法

- 软件: 忙等待、复杂
- 硬件: 忙等待(一直就绪, 不挂起或阻塞)

临界区的互斥访问

- 信号量
- 互斥锁

信号量

- 系统中某种资源的数量
- 初始定义: semaphore x = ?
 - 互斥: S = 1
 - 同步: S = 0 / n S > 0: S个资源可用 S=0: 无资源可用 S<0: 因请求该资源被阻塞的进程个数

P/V操作的含义

- P (S) 申请资源
- V (S) 释放资源
- 操作: P/V 数量一致
 - 互斥: 先P后V

- 同步：先V后P

读写问题（读者优先）

```
semaphore mutex = 1; // 读者优先
int readcount = 0;
semaphore w = 1;
reader(){
    while (1) {
        P(mutex);
        readcount++;
        if (readcount == 1)
            P(w);
        V(mutex); // 保证多个读者同时访问

        READ(); // 读文件

        P(mutex);
        readcount--;
        if (readcount == 0)
            V(w); // 没读者读，写者可以写
        V(mutex);
    }
}
writer(){
    while (1) {
        P(w);
        WRITE(); // 写文件
        V(w);
    }
}
```

读写问题（写者优先）

```
semaphore mutex = 1;
int readcount = 0;
semaphore w = 1;
semaphore rw = 1;
writer(){
    while (1) {
        P(w);
        P(rw);
        WRITE(); // 写文件
        V(w);
        V(rw);
    }
}
reader(){
    while (1) {
        P(w);
```

```
P(mutex);
    readcount++;
    if (readcount == 1)
        P(rw);
V(mutex);
V(w);

READ(); // 读文件

P(mutex);
    readcount--;
    if (readcount == 0)
        V(rw);
V(mutex);
}
```

生产者消费者问题

```
semaphore mutex = 1;
int full = 0;
int empty = n;
// Producer
P(empty);
P(mutex);
    add buffer;
V(mutex);
V(full);
// Consumer
P(full);
P(mutex);
    remove buffer;
V(mutex);
V(empty);
```

引起死锁的四个特征（死锁：多个进程无限期等待永不发生事件）

- 互斥
- 非抢占
- 循环等待
- 持有并等待

克服死锁

- 死锁预防（破坏互斥很难）
- 死锁避免（永不进入不安全状态）
- 死锁检测与解除（终止进程、资源抢占）

4 内存管理

装入（编译 - 链接 - 装入）

逻辑地址（取决于机器的地址长度） -> 绝对地址（取决于内存大小）

- 绝对装入
 - ， 执行前转为物理地址
 - 优点: 简单
 - 缺点: 不适于多道程序系统
- **可重定位装入**: 装入时再修改每个重定位地址项，装入时再根据所定位的内存地址去修改每个重定位地址项，添加相应偏移量（重定位寄存器）
 - 静态: 装入时转换为物理地址
 - 优点: 容易
 - 缺点: 再定位后不能移动
 - 动态: 执行语句再定位
 - 优点: 不必连续存放在内存
 - 缺点: 比较复杂

碎片

- 内碎片: 某进程分配的内存区域部分没有用上
- 外碎片: 某些空闲分区难以利用

存储管理方案

- 连续内存分配: 一个**连续的内存空间**
 - 单一连续: 有内碎片
 - 分区分配:
 - 固定分区（分区大小相等/不等）: 有内碎片
 - 动态分区: 有外碎片 分区保护: 界限寄存器、存储保护键
- 离散内存分配方式:
 - 分页存储管理
 - 段式存储管理（解决页式的碎片问题）
 - 段页式存储管理（一个段表管理逻辑地址，一组页表管理物理地址）
- 虚拟存储器（带有中断机制）

动态分区分配——分区分配算法

- 最先适配算法（地址递增）
 - **从头查找**
 - 实质
 - 尽可能利用存储区域空闲区，高地址保存较大空闲区，有大要求容易满足
 - 优点
 - 简单
 - 合并相邻空闲区容易
 - 缺点
 - 内存分配不均匀
 - 每次都从头找
- **循环最先适配算法**

- **上次分配分区开始**
- 优点
 - 时间性能好，分区更均匀
- 缺点：
 - 但较大空闲不易保留
- 最佳适配算法（长度递增）
 - **最小分区**
 - 分区表按照空闲区从小到大排序
 - 优点: 较大的空闲分区可以保留
 - 缺点
 - 外碎片多
 - 不好合并空闲区
- 最坏适配算法（长度递减）
 - **最大分区**
 - 空闲区大到小排序
 - 优点: 只用找一次，要么成功，要么失败
 - 缺点: 剩余分区会越来越少，无法运行大程序

内存扩充——交换

- 整体交换：整个进程为单位 目的解决内存紧张，进一步提高内存利用率。
- 部分交换：以分页、分段交换为基础 目的是支持虚拟存储系统
- 优点：增加并发程序数目，提供适当的响应时间

简单分页存储管理

- 一个进程有一个页表（图中慢表）
- 页表项：页号 || 块号（页框号） 页表项（在页表中）地址： $F + P * \text{页表项长度（大小）}$ 一个页表项对应一个页面
- 快表TLB
 - 独立于内存的高速缓冲存储器（Cache）

- 缩短查找时间

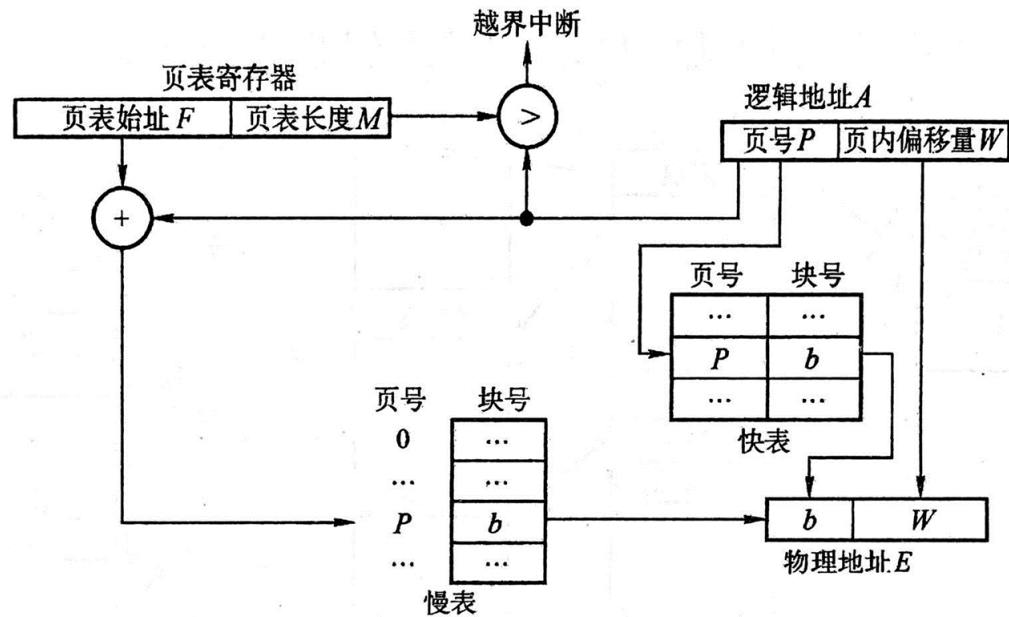


图 3.10 具有快表的地址变换机构

- 逻辑地址 A : 页号 P + 页内偏移量 W $P = A / L, W = A \% L$
- 页面数: 2^P , 页面大小: 2^W
- 物理地址 E : 页框号 $b * \text{页面大小} L + \text{页内偏移量} W$

逻辑地址 A

↓

$[P = A / L] \rightarrow \text{页表查找} \rightarrow \text{页框号 } b$

$[W = A \% L]$

↓

物理地址 $E = b * L + W$

- 页表用于记录逻辑页与物理块的映射，存放于**内存**，首地址在 **PCB**；
- 位图用于表示页框使用情况，存于**内存**，是一种高效的分配表；例如：若有 1M 页框，则位图大小为：

$$\frac{2^{20}}{8} = 128KB$$

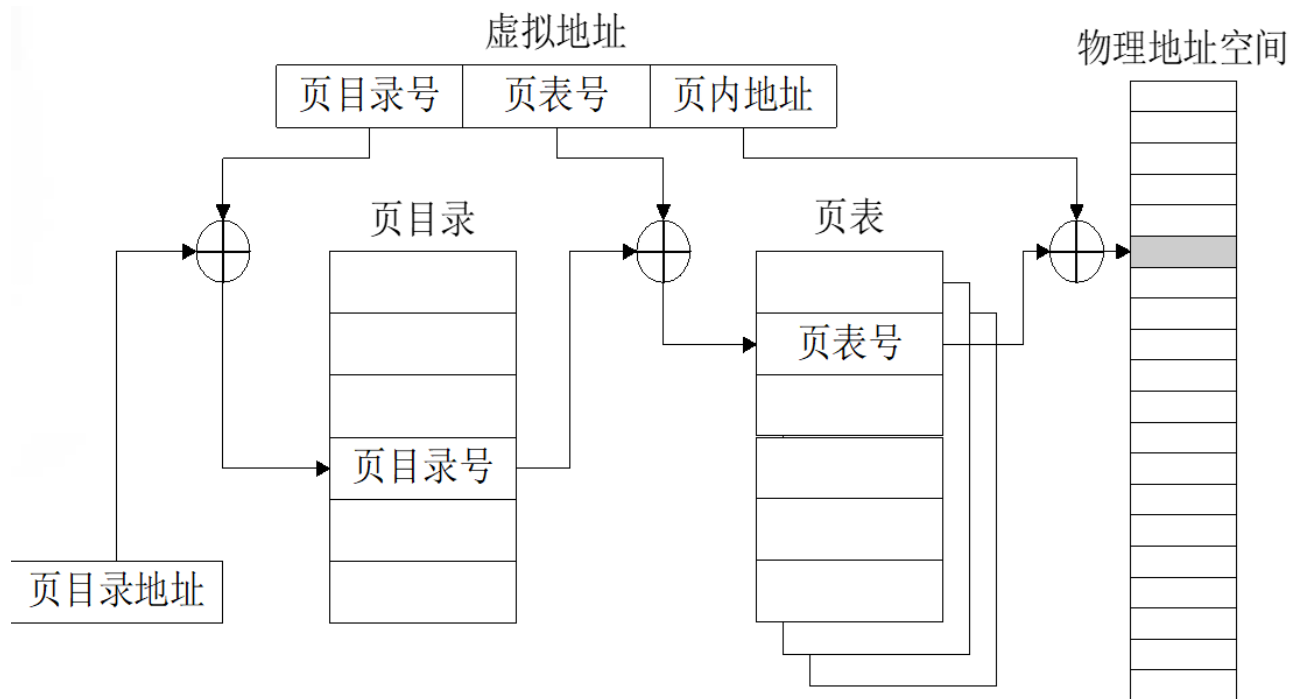
虚拟存储器

- 具有请求调入功能和置换功能，能从**逻辑**上对**内存容量加以扩充**的一种存储器系统
- 特征：
 - 不连续性
 - 部分交换
 - 大空间
- 最大容量由**计算机的地址结构**决定，目的为**扩充主存容量**
- 实际容量**内存和外存交换区容量之和**
- 最大页数/段数以及段长度由**逻辑地址结构**决定

虚拟内存管理——请求页式管理机制

只在页面需要时，才将其载入内存

- 缺页中断过程
 1. 执行指令，产生缺页中断
 2. CPU保存状态，操作系统接管
 3. 主存向外存请求调页
 4. 内存有空闲页框？
 - 有就调入
 - 没有就**页面替换算法**替换
 5. 修改页表
 6. 恢复现场，启动进程，继续执行
- 优点：节省内存
- 二级页表地址映射：



5种页面置换算法

- FIFO先进先出
 - 简单
 - 抖动现象 -> 频繁换入换出
 - Belady现象：给更多的页但不一定会有更好的效果
- OPT最佳算法(理想化)
 - **最长时间内不被访问**
 - 预测未来用未来不再使用的，在离当前最远位置上出现的
- LRU最近最少使用
 - **自上次访问，最少使用的**
 - 利用历史预测将来，置换最长时间没有使用的页
 - 性能接近最佳算法
- LFU最不常用
 - **从某时候，使用次数最少**
 - 每一页都设置访问计数器
 - 页中断时换出计数最小的，并**所有计数清零**

- 若没有相同的，跟FIFO相同，一直都缺页一直清零，结合FIFO换出
- clock轮转算法
 - LRU和FIFO的折衷
 - 采用一个指针，从当前指针位置开始按地址先后检查各页，寻找use=0的页面作为被置换页，并将指针经过的页修改为use=0，最后指针停留在**被置换页的下一个页**
- 抖动（选的页面置换算法不好）：刚换出的要换入，刚换入的要换出

5 文件管理

文件

- 计算机硬盘为载体存储在计算机的信息集合
- 文件 = PCB + 文件体

文件系统

文件、管理文件的软件及数据结构

- **实现对文件的按名存取**

文件的逻辑结构、物理结构

- 逻辑结构
 - 流式文件：
 - 构成文件的基本单位是字符，文件是有逻辑意义的、无结构的一串字符的集合
 - 提供很大的灵活性
 - 记录文件
 - 文件是由若干个记录组成，是一个固定长度记录的序列，每条记录有其内部结构每个记录有一个键，可按键进行查找
- 物理结构--从文件在物理介质上的存放方式
 - 顺序结构 文件信息存放在若干连续的物理块中
 - 优点
 - 简单
 - 顺序存取速度快
 - 所需磁盘寻道次数和存到时间最少
 - 缺点
 - 不利于文件动态增长
 - 外碎片
 - 链接结构 每个块末尾隐含指向下一个块的指针，-1标识EOF
 - 优点
 - 提高磁盘空间利用率，不存在外碎片
 - 文件插入删除
 - 缺点
 - 不能**随机存取**
 - 链接指针占用一定空间
 - 索引结构 文件信息存放在若干不连续物理块中，系统为每个文件建立一个专用的数据结构索引表，并将这些块的块号存放在该索引表中 一个索引表就是磁盘块地址数组，第i个条目指向文件的第i块

- 优点
 - 能顺序也能**随机**
 - 动态增长、插入删除要求
 - 利用外存
- 缺点
 - 较多寻道次数和寻道时间
 - 索引表本身带来系统开销：内外存空间、存取时间

存储介质	磁带	磁盘		
物理结构	连续结构	连续	链接	索引
存取方式	顺序存取	顺序	顺序	顺序
		随机		随机

- 顺序存取 → 顺序文件结构更快;随机存取 → 索引文件结构更快

综合索引方式

- UNIX系统：流式文件，成组链接，综合索引
- 索引表中有直接地址直接指向数据块的，也有一级间接地址指向下一级索引表
- 支持大小文件，查找速度快

磁盘空闲空间管理方法

1.数据结构：

- 空闲块表
- 空闲块链表

2.管理方法

- 位示图（开销大）
- 成组链接法（查找效率高）

成组链接法

- 下一组 空闲块总数 和 物理盘块号 (递归)
- 优点：克服了空闲链表太长，分配和回收盘块简单

目录文件

- 管理文件
- 将文件目录以文件形式保存在外存
- 目录项：构成文件目录的项目（FCB：文件名 + 存放地址）

采用目录文件的目的

高效性、重命名、逻辑组

目录的改进方法

- 目录项分解：

举例：

- 一个FCB有48个字节，符号目录项占 8字节，文件名6字节，文件号2字节，基本目录项占 $48-6=42$ 字节。假设，物理块大小512字节，目录文件有128个目录项。
- 改进前：1个物理块含 $512/48=10$ 个FCB，则该目录文件占13个物理块；
- 改进后：1个物理块含 $512/8=64$ 个符号目录项或 $512/42=12$ 个基本目录项，则该目录文件符号文件占2块，基本文件占11块。

查找一个文件的平均访盘次数

改进前： $(1+13)/2=7$ 次

改进后： $(1+2)/2 + 1 = 2.5$ 次

减少了访问硬盘的次数，提高了检索速度

常用的目录结构

- 多级目录（命名冲突）
- 树形目录
- 哈希
- B+树

文件操作中Create()/Open()

- Create()
 - 创建文件的FCB，建立必要的存储空间，分配空FCB，根据提供的参数及需要填写有关内容返回一个文件描述
 - 执行过程
 - 检查参数合法性
 - 检查同一目录下有无重名文件
 - 在目录中有无空闲位置
 - 填写目录项内容
 - 返回
- Open()
 - 文件使用前都需要先打开，即把FCB送到内存
 - 执行过程
 - 文件路径名查目录，找到FCB主部
 - 根据打开方式、共享说明和用户身份检查访问合法性
 - 根据文件号查系统打开文件表，看文件是否已被打开
 - 是：共享计数+1

- 否：将外存中的FCB主部等信息填入系统打开文件表空表项，共享计数 = 1

磁盘

- 外存
- 磁带：顺序
- 磁盘：随机（可以顺序）
- 光盘

影响磁盘访问的因素

- 寻道时间
- 旋转延迟
- 数据传输

磁盘调度时间

一次访盘时间 = 寻道时间 + 旋转延迟时间 + 存取时间

磁盘调度算法

- 先来先服务(FCFS)
 - 按访问请求到达的先后次序服务
 - 优点：公平简单
 - 缺点：效率不高，加了时间
- **最短寻道时间**
 - 优先选择距当前磁头最近的访问请求进行服务，主要考虑寻道优先
 - 优点：改善了磁盘平均服务时间
 - 缺点：造成某些访问请求长期等待不到服务（饥饿）
- 扫描算法(电梯算法，双向扫描)
- 单向扫描算法